10

Immagine processo UNIX

Process Structure:
- Metadati (pid, stato, ...)
- Non swappable
- → Process table nel kernel

User Structure:
- Dati processo in esecuzione
- cpu registri, fd, handler, ...
- Swappable

Text (codice):
Codice binario ed eseguibile (sola lettura)

Ecex():
- Sovrascrive tutto
- Avvia nuovo programma

Stack e Heap:
- Aree dinamiche
- Stack creasce per gestire record di attivazione delle funzioni e le variabili locali
- Heap allocazione dinamica della memoria

Fork():
- Processo figlio,
- Copia dati + stack dal padre.
- Spazi di memoria divisi

11

Tipologie di Scheduling

Mid-Term Scheduling:
Ram → Dischi
Dischi->Ram

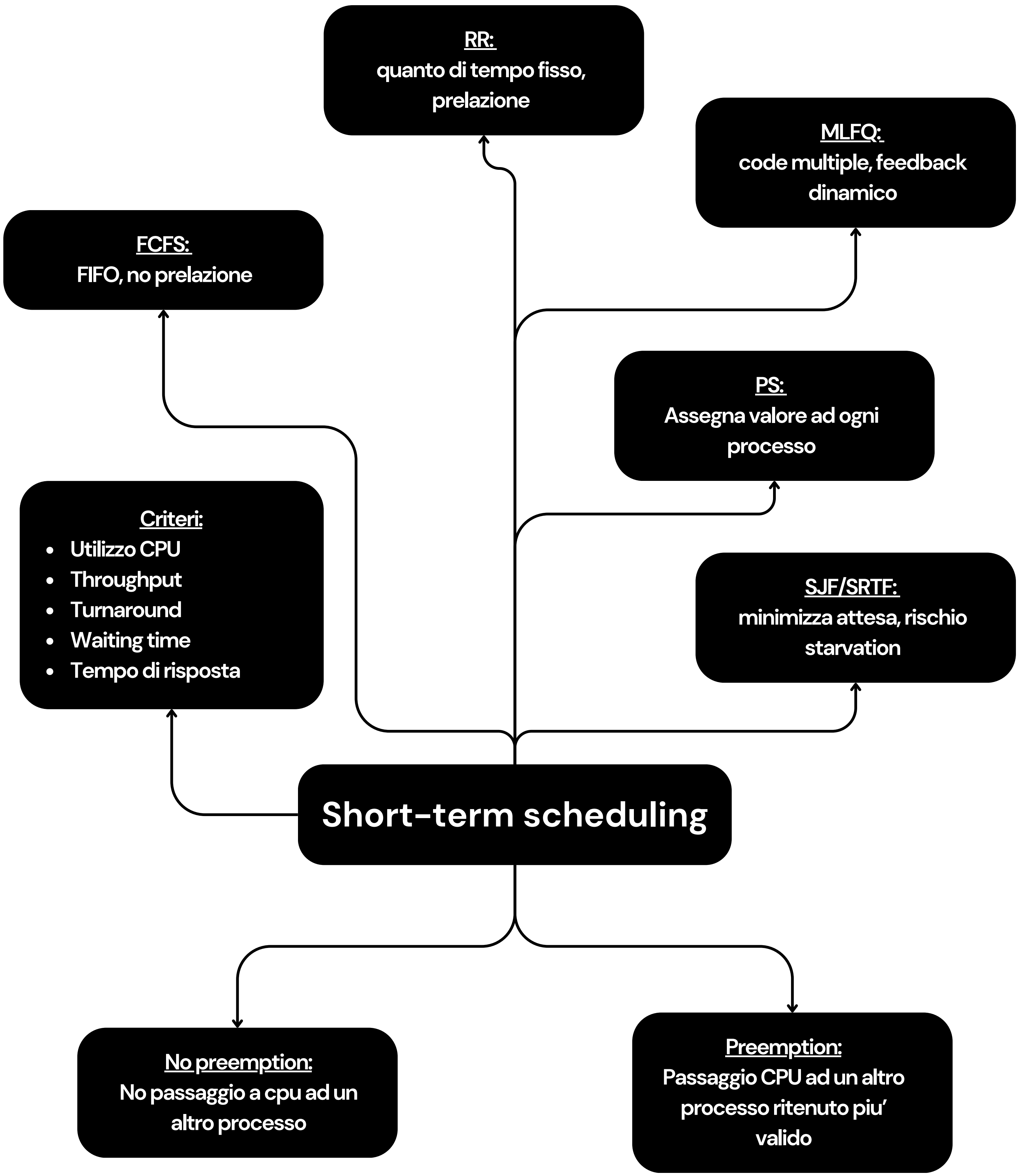
Long-Term Scheduling:
- Dischi→ Ram
- Creazione processi
- Gestione Multiprogrammazione

Short-Term Scheduling:
Ready queue → CPU

Scelta dell'ordinamento → Scheduler
Meccanismo SW per cambio contesto → Dispatcher

"Il dispatcher è il modulo del kernel che assegna concretamente il controllo della CPU al processo che è stato appena selezionato dallo scheduler. Mentre lo scheduler implementa la politica decidendo 'chi' è il prossimo a dover essere eseguito, il dispatcher è il meccanismo che esegue materialmente il cambio."
"Per dare il controllo al nuovo processo, il dispatcher deve compiere tre operazioni fondamentali:

1. Effettuare il Context Switch: salva lo stato del processo uscente e ripristina quello del processo entrante.
2. Cambiare la modalità di esecuzione: passa dalla modalità kernel (usata per il context switch) alla modalità utente.
3. Eseguire il salto (Jump): aggiorna il Program Counter per riprendere l'esecuzione del programma utente dal punto esatto in cui era stata interrotta."


RR:
quanto di tempo fisso, prelazione
MLFQ:
code multiple, feedback dinamico
FCFS:
FIFO, no prelazione
PS:
Assegna valore ad ogni processo
Criteri:
Utilizzo CPU
Throughput
Turnaround
Waiting time
Tempo di risposta
SJF/SRTF:
minimizza attesa, rischio starvation
Short-term scheduling
No preemption:
No passaggio a cpu ad un altro processo
Preemption:
Passaggio CPU ad un altro processo ritenuto piu' valido

13

Protezione file UNIX

12 bit

XXX-XXX-XXX-XXX

3 Bit Speciali

setuid: esegue con privilegi del proprietario

setgid: esegue con privilegi del gruppo

Sticky bit: solo il proprietario può cancellare in /tmp

9 Bit Standard:

rwx: read, write, execute

- 1-3: Owner
- 4-6: Group
- 7-9: Others

14

Metodi di accesso e allcazione

Accesso diretto: specifica numero blocco

Accesso sequenziale: scorre tutti i record precedenti

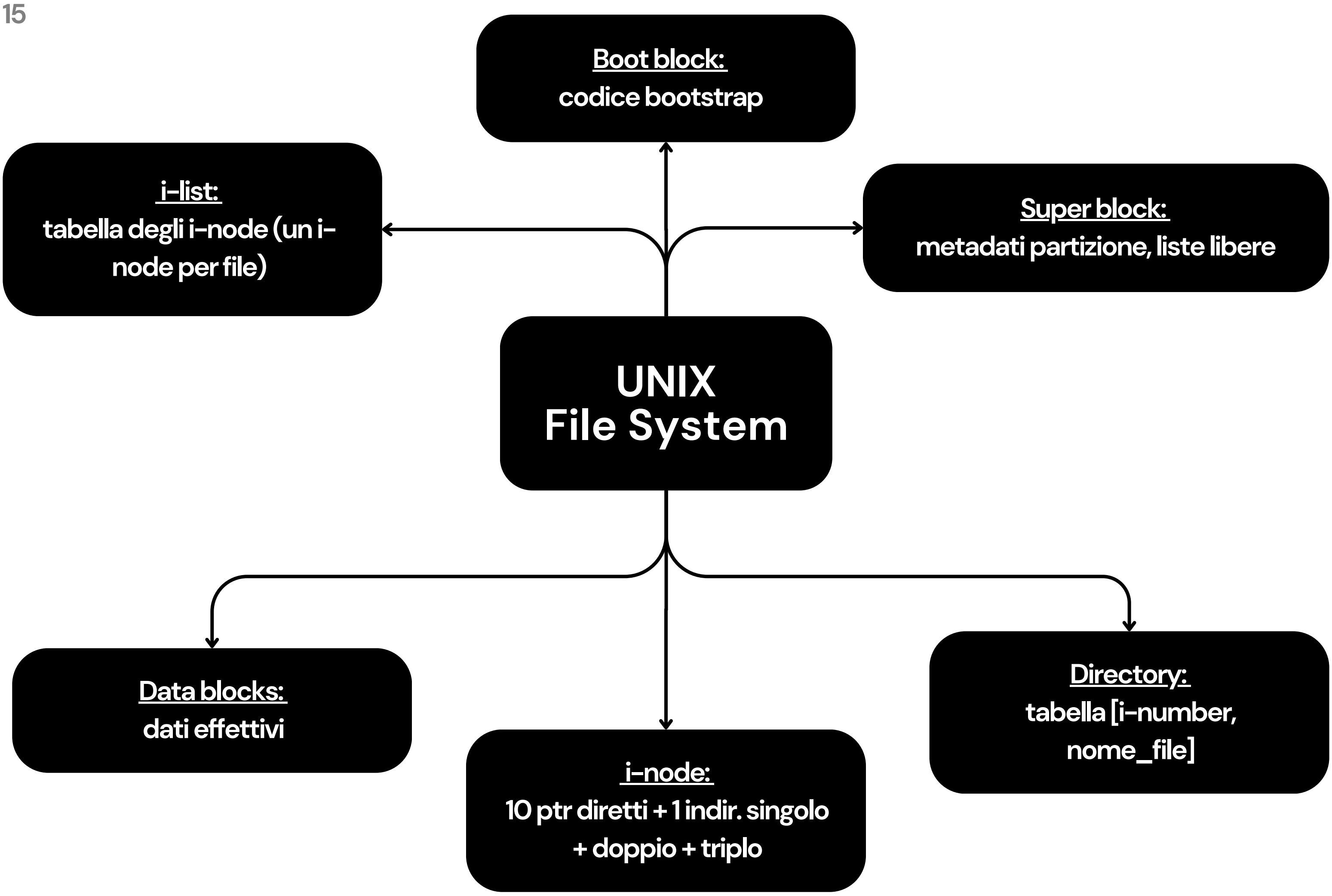
Accesso a indice: struttura ausiliaria con puntatori

A indice: supporta accesso diretto, spreco per file piccoli

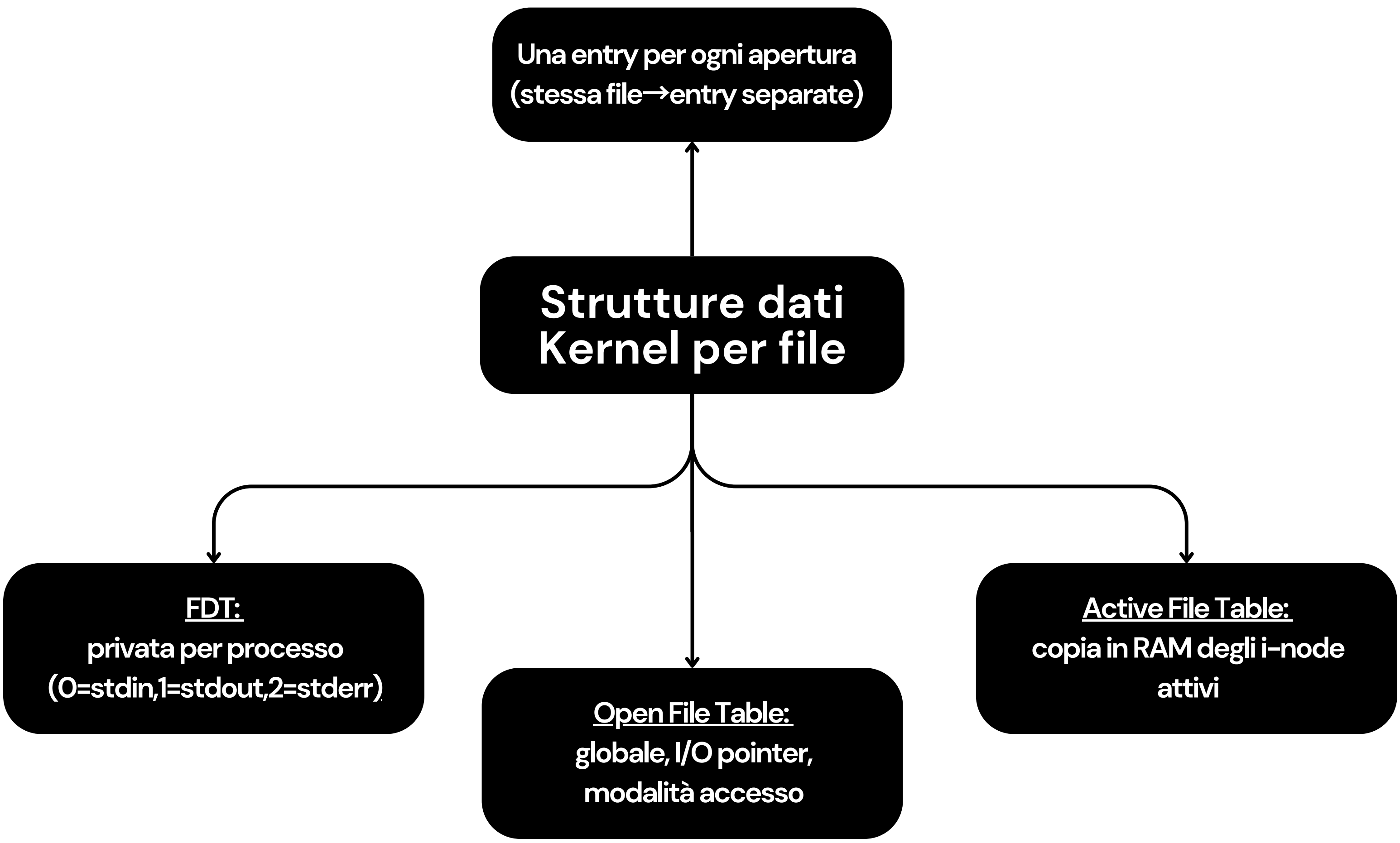
Concatenata: no framm. esterna, accesso diretto lento

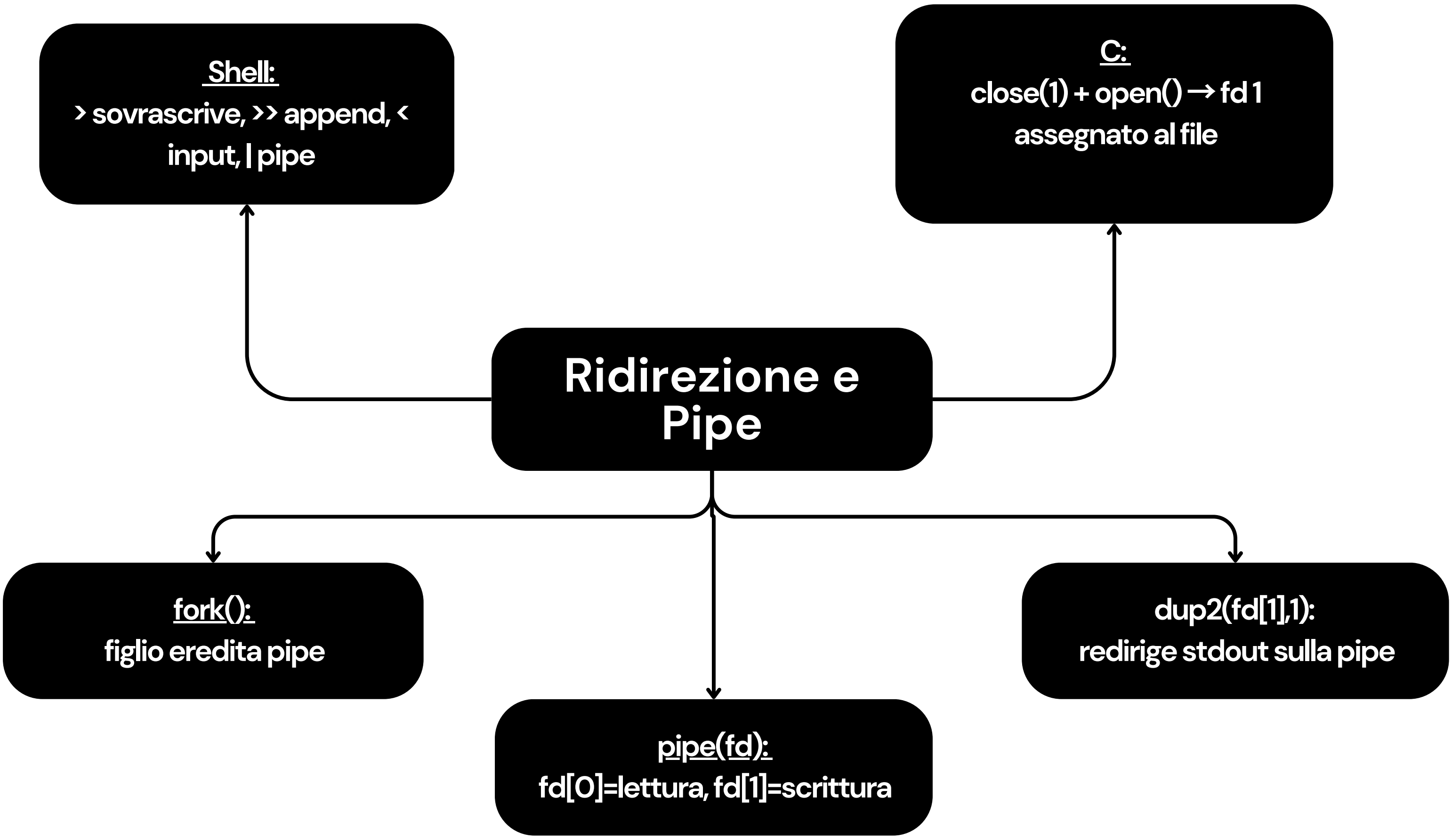
Contigua: veloce, ma frammentazione esterna

15


UNIX File System
Boot block:
codice bootstrap
i-list:
tabella degli i-node (un i-node per file)
Super block:
metadati partizione, liste libere
Data blocks:
dati effettivi
i-node:
10 ptr diretti + 1 indir. singolo + doppio + triplo
Directory:
tabella [i-number, nome_file]

16


Strutture dati Kernel per file
Una entry per ogni apertura (stessa file→entry separate)
FDT:
privata per processo (0=stdin,1=stdout,2=stderr)
Open File Table:
globale, I/O pointer, modalità accesso
Active File Table:
copia in RAM degli i-node attivi


Ridirezione e Pipe
Shell:
> sovrascrive, >> append, < input, | pipe
C:
close(1) + open() → fd 1 assegnato al file
fork():
figlio eredita pipe
pipe(fd):
fd[0]=lettura, fd[1]=scrittura
dup2(fd[1],1):
redirige stdout sulla pipe

18

Allocazione in Ram

- ALLOCAZIONE CONTIGUA (MODELLI STORICI)
  - <u>MFT:</u> partizioni fisse, framm. interna
  - <u>MVT:</u> partizioni dinamiche, framm. esterna → compattazione
  - <u>Partizione singola:</u> monoprogrammazione
- ALLOCAZIONE NON CONTIGUA (MODELLI MODERNI)
  - <u>Paginazione:</u> frame fissi, no framm. esterna, tabella pagine
  - <u>Segmentazione:</u> blocchi variabili, framm. esterna
  - <u>Segm. paginata:</u> ibrido (Intel, Linux)

19

Memoria Virtuale

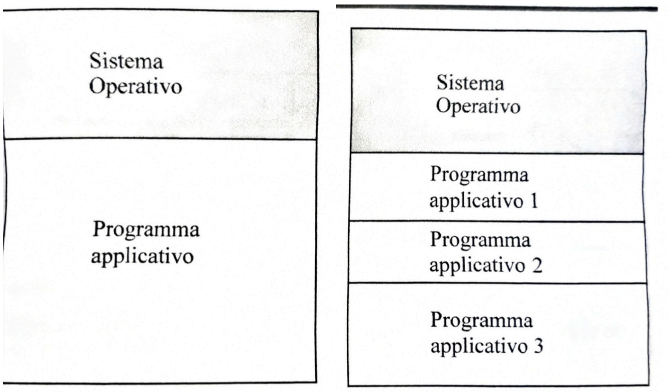
- Processo può essere più grande della RAM fisica
- Spazio logico mappato su disco
- Solo le pagine necessarie sono in RAM
- Implementata tramite paginazione su richiesta

20

Paginazione su richiesta

- <u>Kernel:</u> verifica liceità → cerca frame libero
- Carica solo pagine strettamente necessarie all'avvio
- Se no frame libero → sceglie vittima (swap-out)
- <u>Page fault:</u> MMU rileva pagina non in RAM → interrupt
- <u>Carica pagina richiesta → aggiorna tabella → riprende</u>

1–2



Organizzazione della memoria in un sistema batch monoprogrammato

Organizzazione della memoria in un sistema batch multiprogrammato

1–2

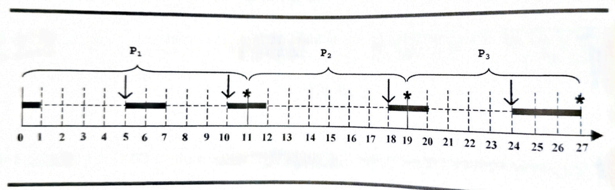


Figura 1.4 Esecuzione sequenziale.

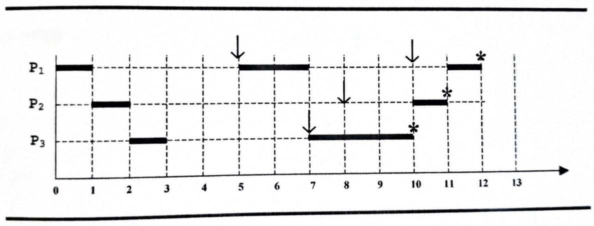


Figura 1.5 Esecuzione in multi-tasking.

DMA

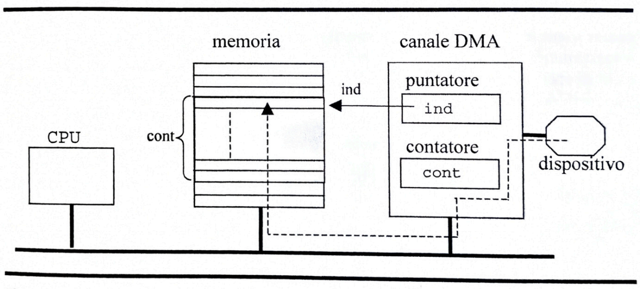


Figura 1.15 Canale di DMA.

1–2

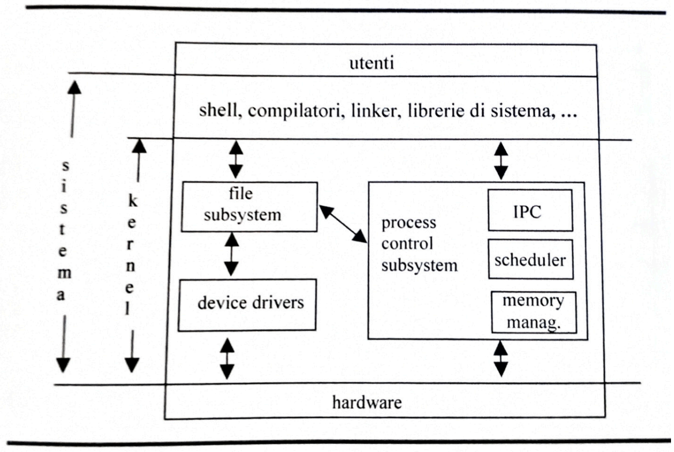


Figura 1.18 Struttura del sistema Unix.

1–2

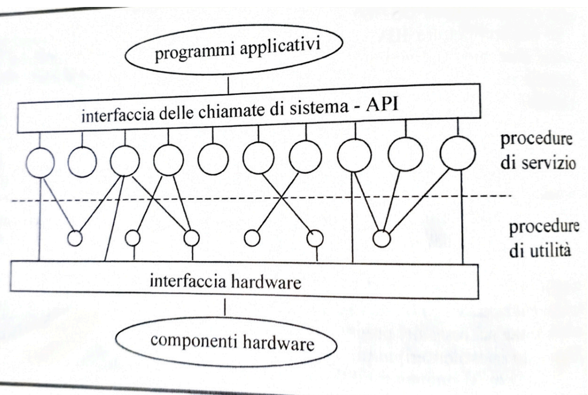


Figura 1.17 Semplice modello strutturale di un sistema modulare.

5

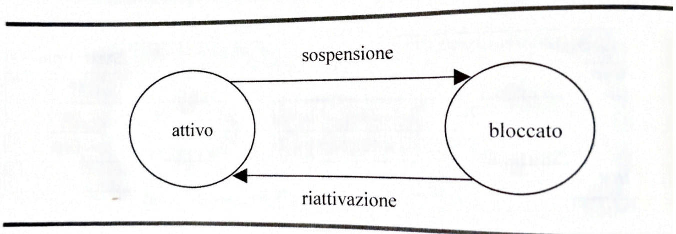
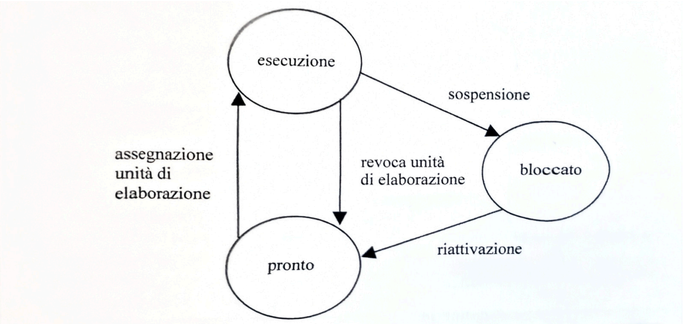


Figura 2.2 Stati di un processo.

5



Stati di un processo, in sistemi in cui il numero di processi supera il numero delle unita' di eleborazione

5

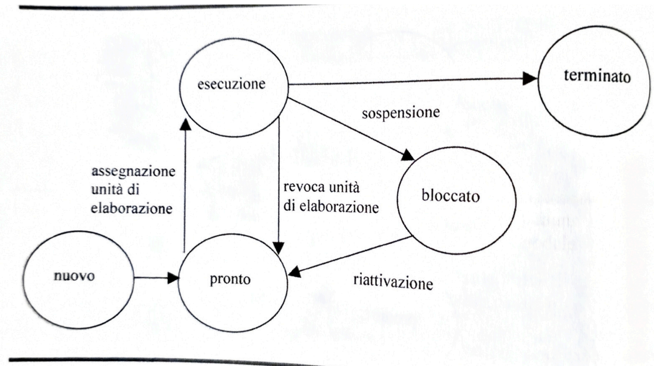
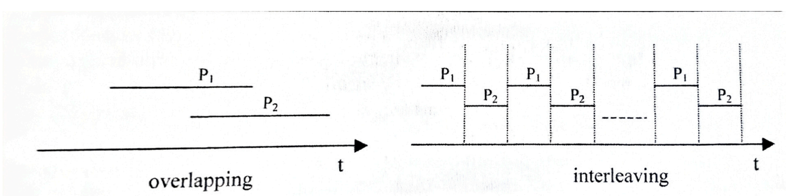


Figura 2.4 Modello a cinque stati.

5



Processi concorrenti

6–7

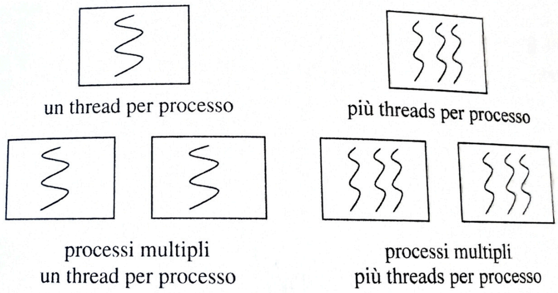


Figura 2.27 Multithreading.

8

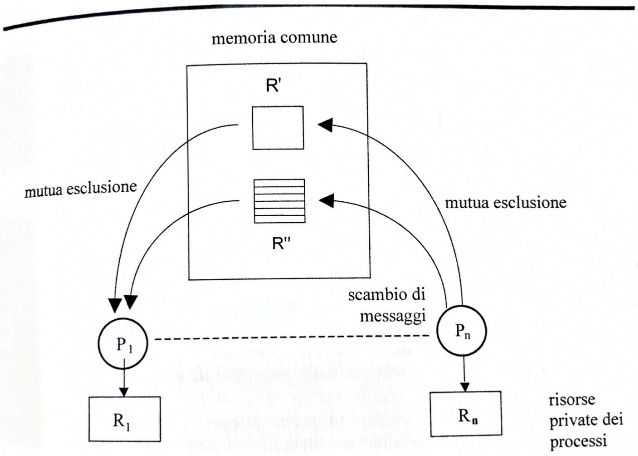


Figura 3.1 Interazioni tra processi in sistemi a memoria comune.

5

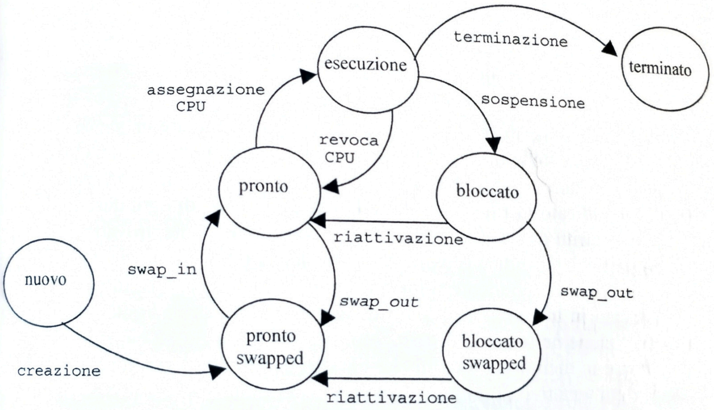


Figura 4.21 Stati swapped di un processo.

11

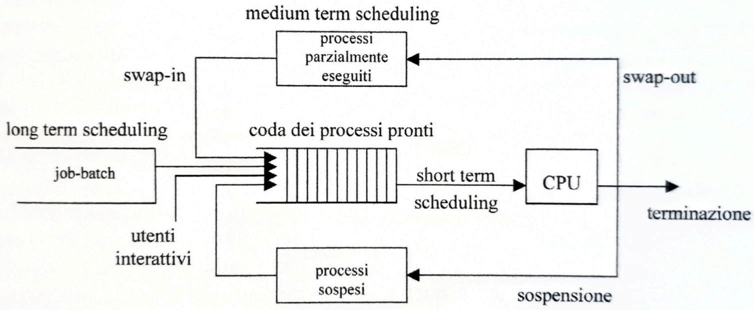


Figura 2.11 Livelli di scheduling.

8

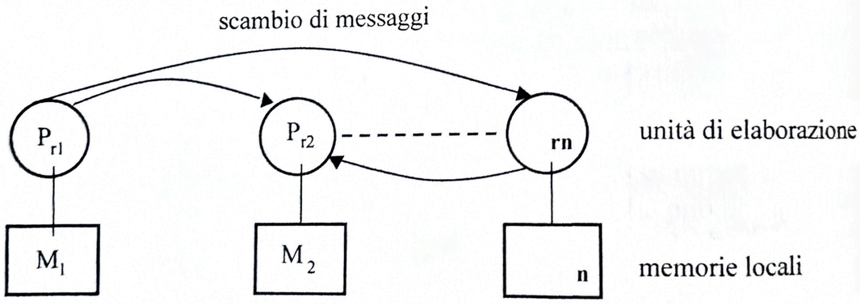


Figura 3.2 Interazioni tra processi in sistemi a memoria locale.

12

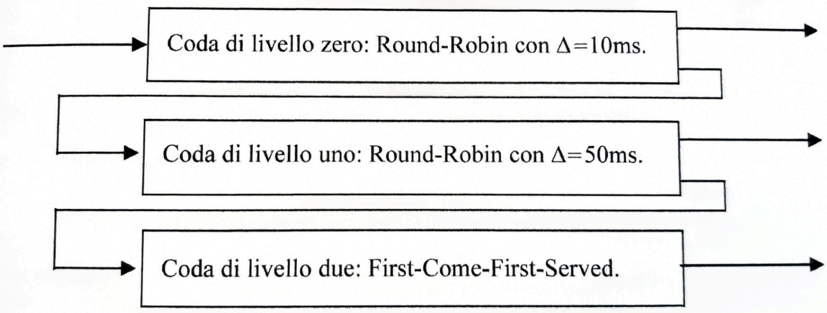


Figura 2.21 Esempio di *multilevel feedback queue*.

11

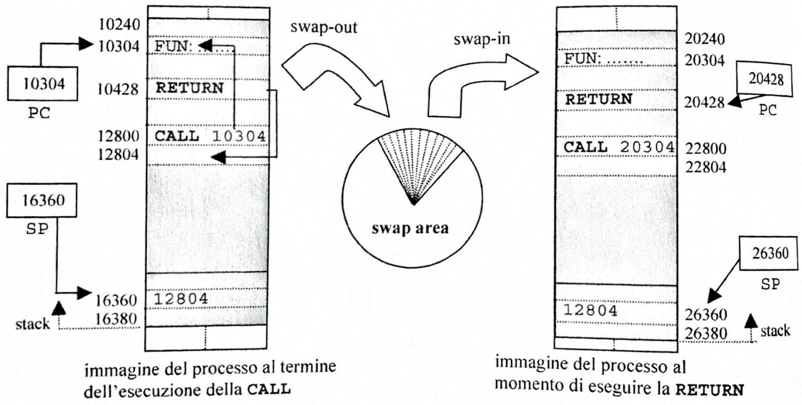


Figura 4.5 Revoca di memoria e successiva riallocazione in una diversa partizione.

10

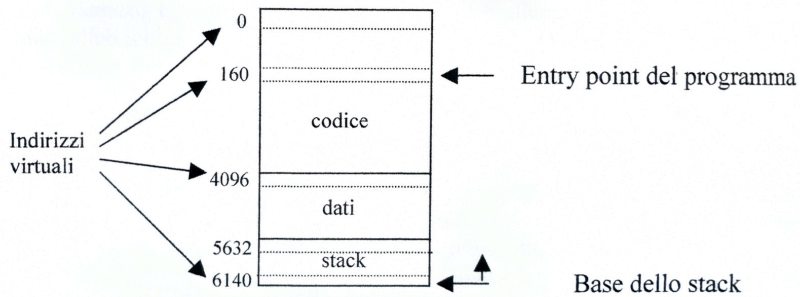
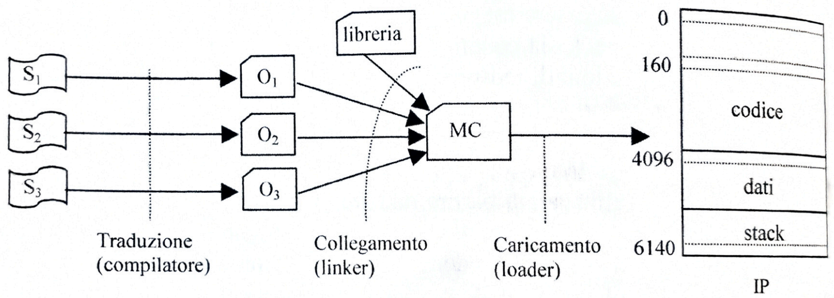


Figura 4.1 Immagine di un processo.

10



S_1, S_2, S_3: moduli sorgente; O_1, O_2, O_3: moduli oggetto;
MC: modulo di caricamento (file eseguibile); IP: immagine del processo

Figura 4.2 Preparazione di un programma per l'esecuzione.

18

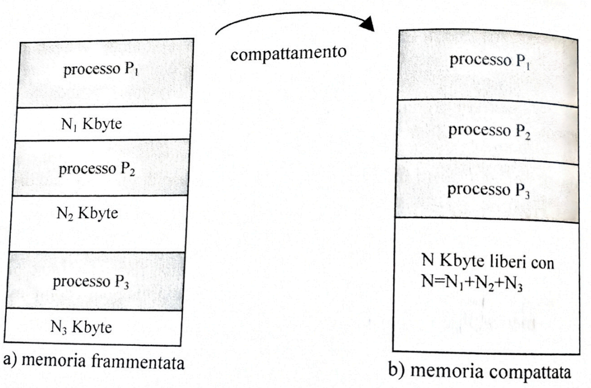
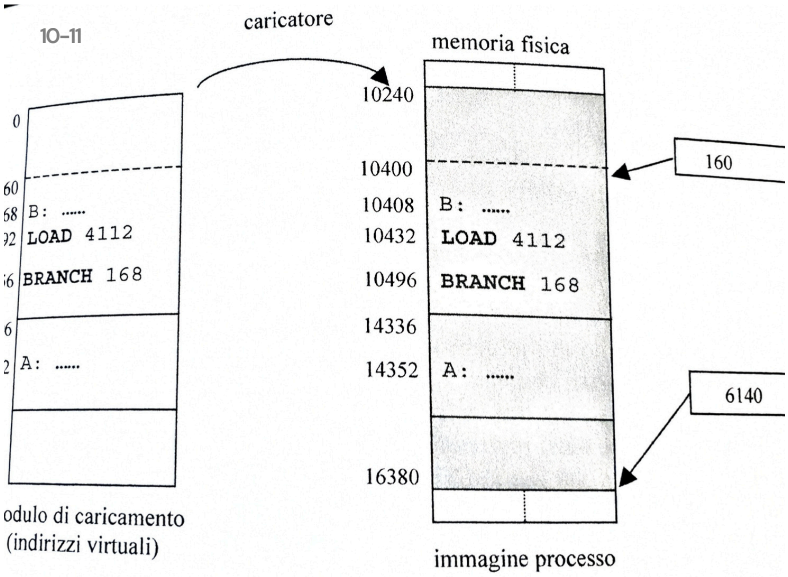


Figura 4.10 Compattamento.

10-11



4.6 Rilocazione dinamica.

8

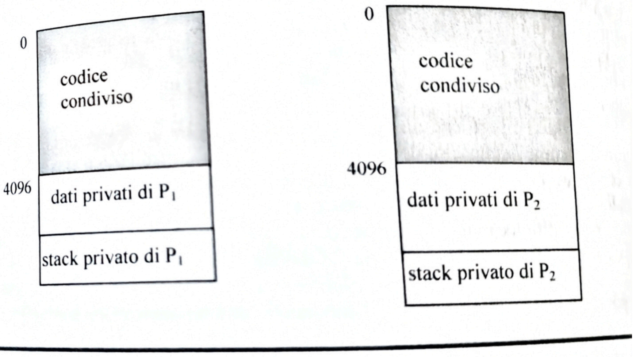


Figura 4.11 Codice condiviso.

18

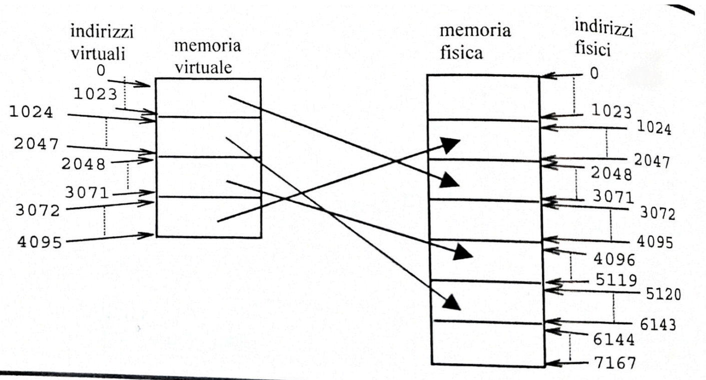


Figura 4.23 Paginazione.

20–21

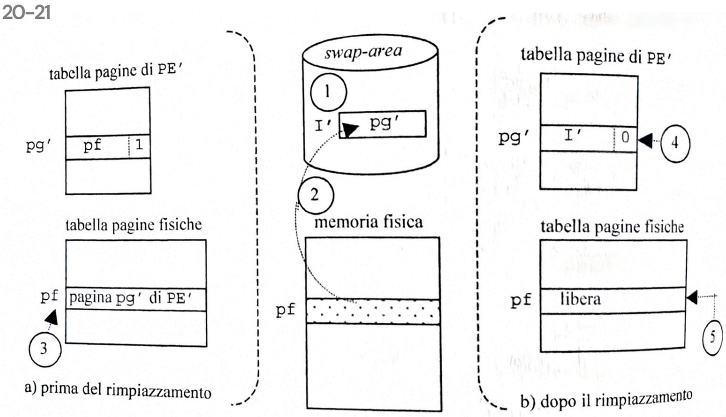


Figura 4.30 Rimpiazzamento delle pagine.

20–21

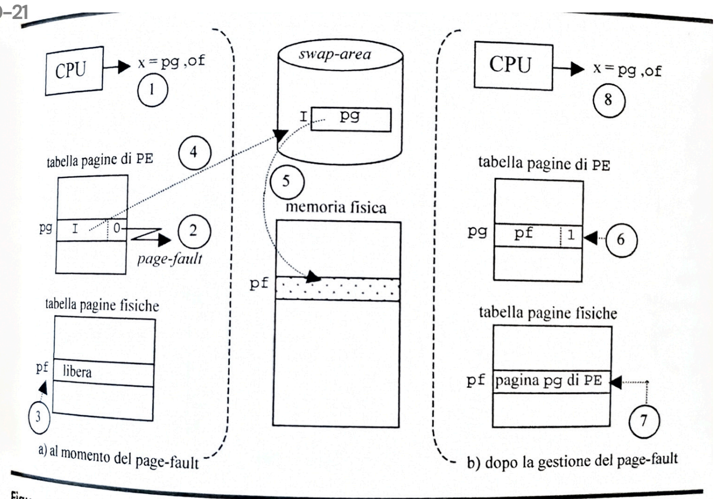


Figura 4.29 Gestione di un page-fault.

18

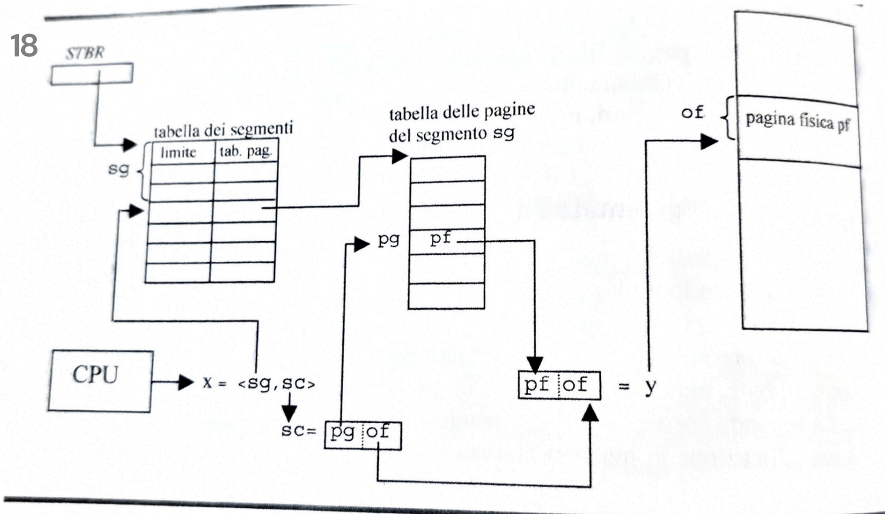


Figura 4.33 Segmentazione paginata.

24

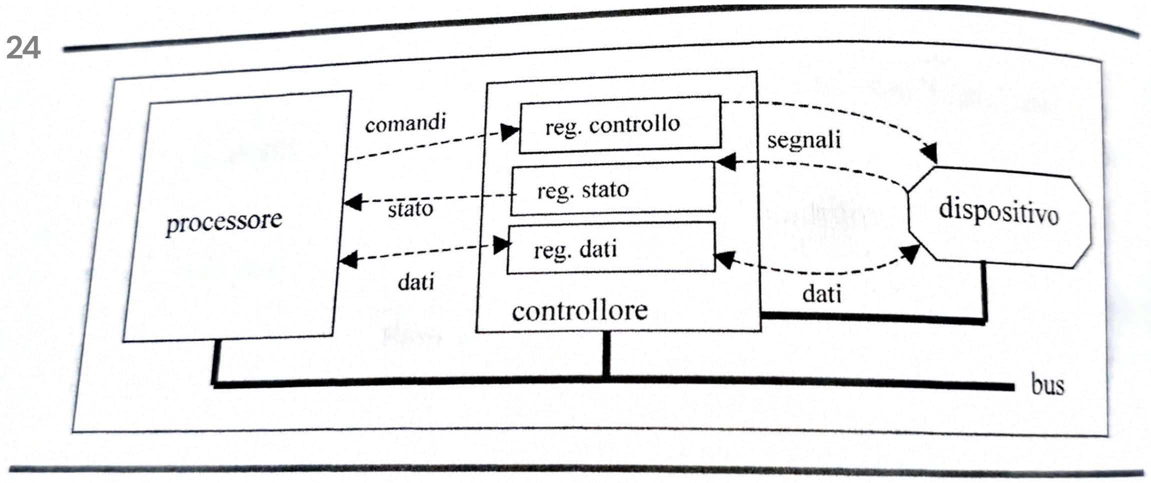


Figura 5.5 Controllore di un dispositivo.

DMA

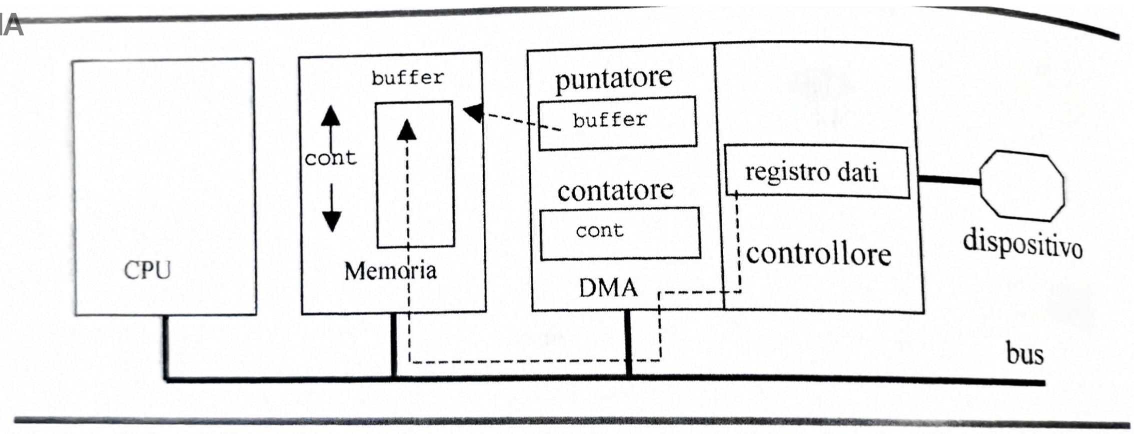


Figura 5.13 Canale di DMA.

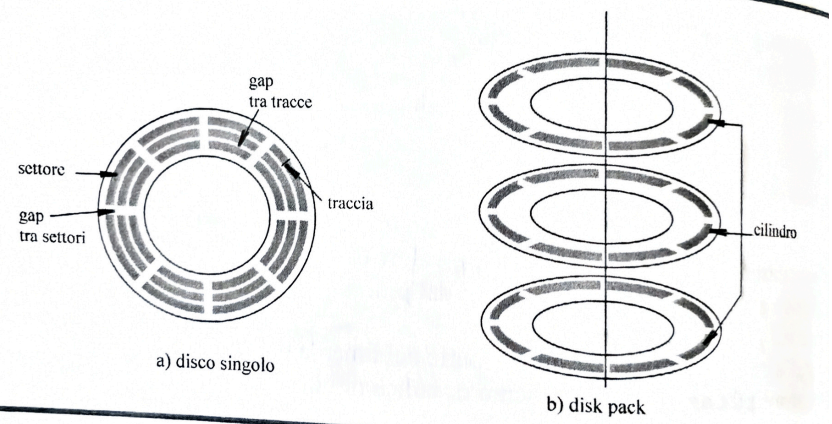


Figura 5.16 Organizzazione fisica dei dischi.

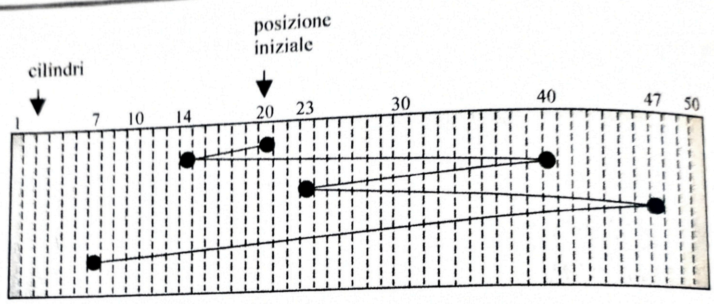


Figura 5.18 Algoritmo di scheduling FCFS.

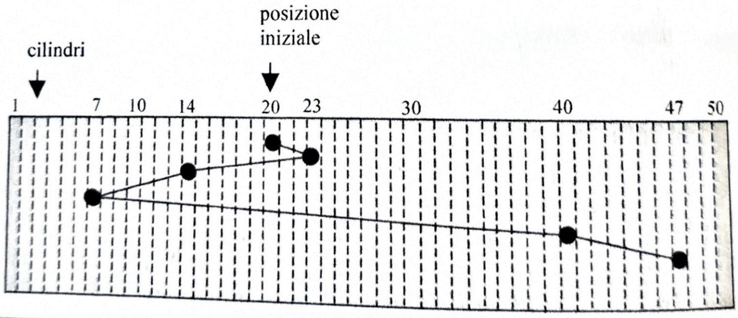


Figura 5.19 Algoritmo di scheduling SSTF.

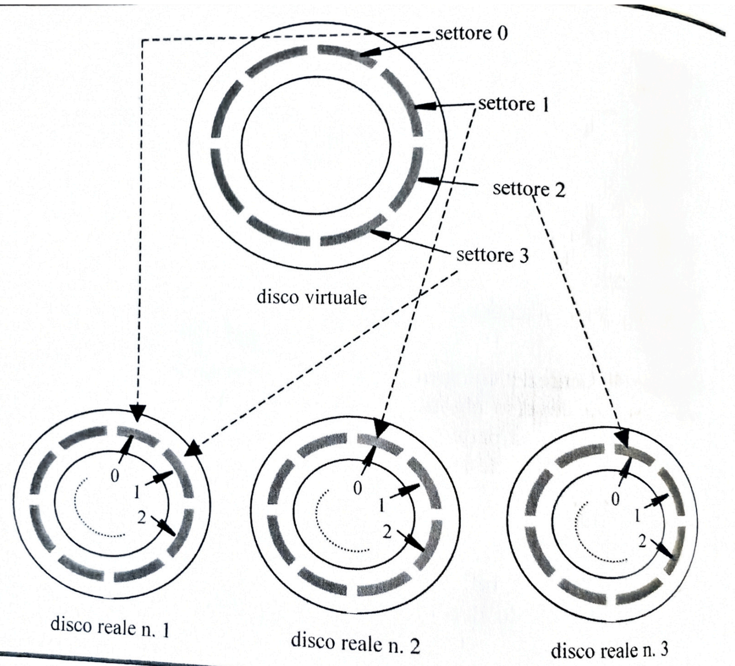


Figura 5.21 Dischi RAID.

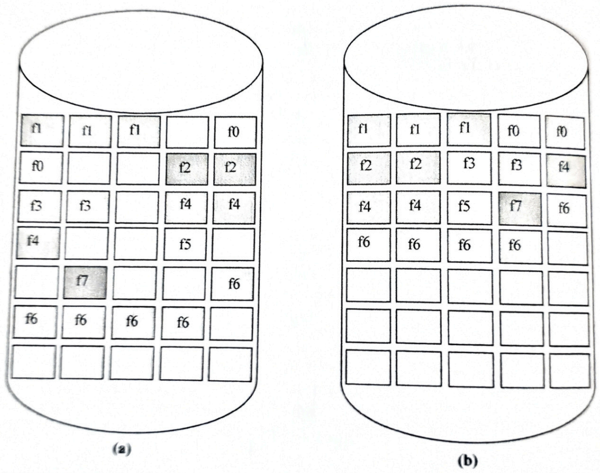


Figura 6.8 Frammentazione dello spazio disponibile su disco (a) e compattamento (b).

14

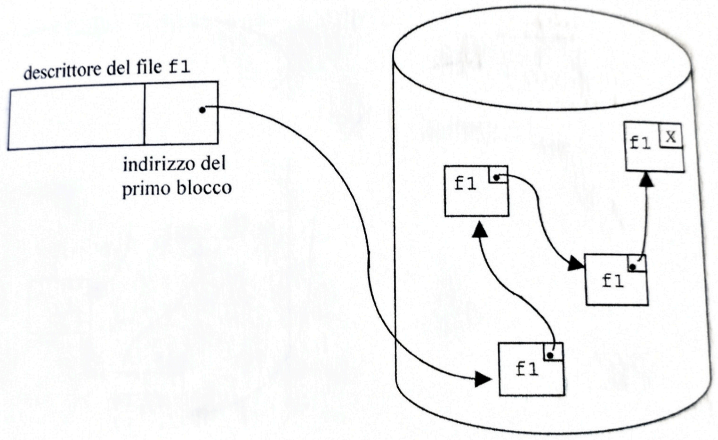


Figura 6.9 Allocazione a lista concatenata.

14

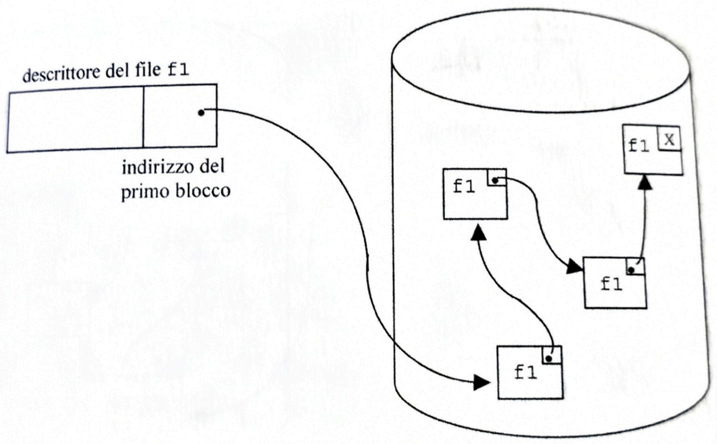


Figura 6.9 Allocazione a lista concatenata.

14

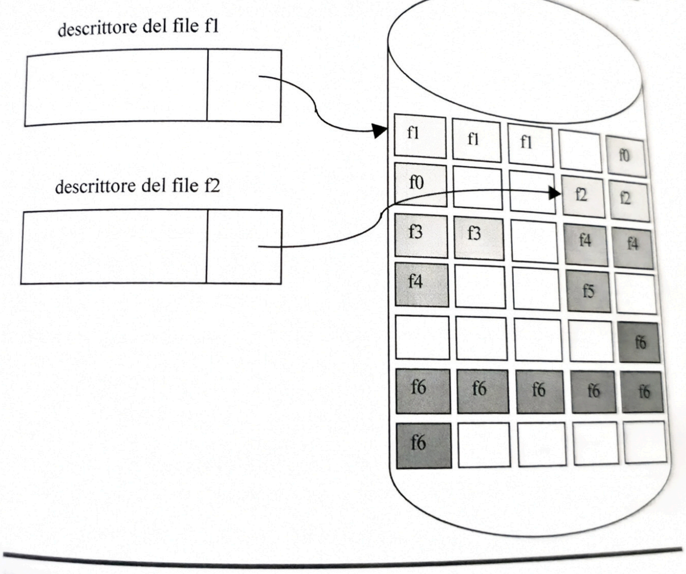


Figura 6.7 Allocazione contigua di file.

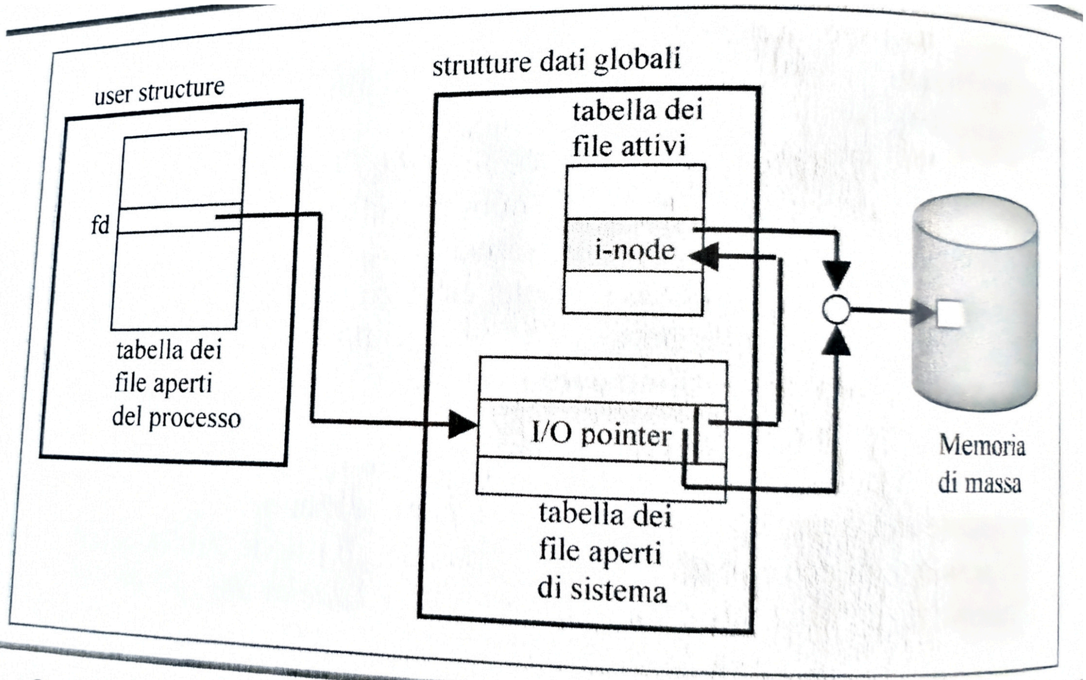


Figura 8.14 Strutture dati per l'accesso a un file.

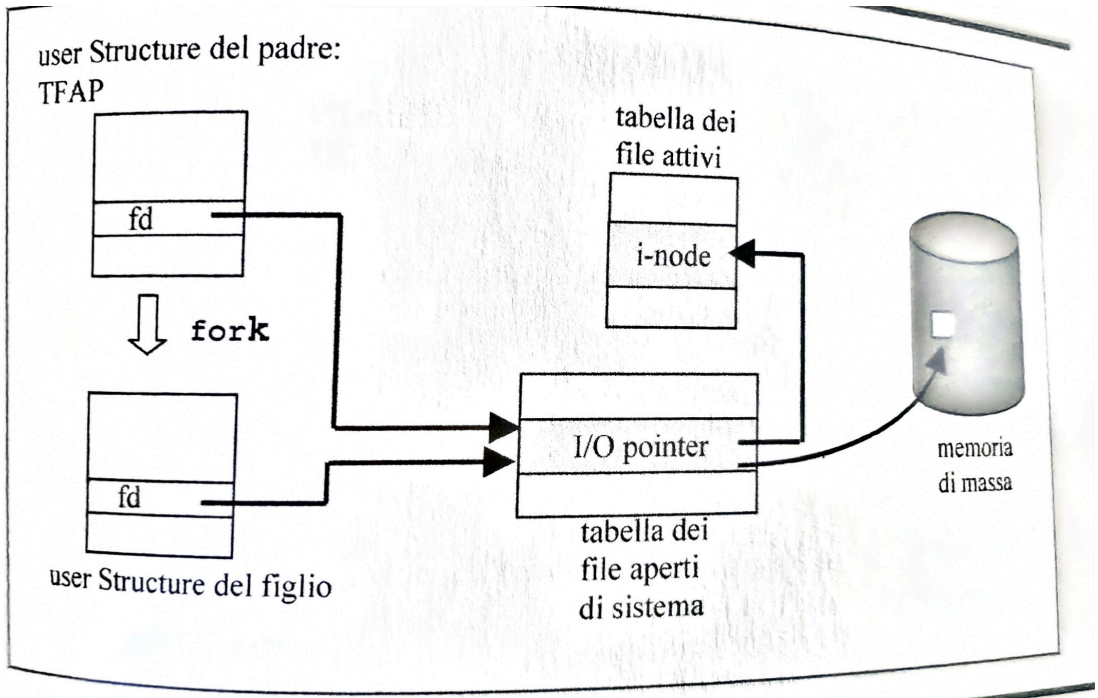


Figura 8.15 Condivisione di I/O pointer tra padre e figlio.

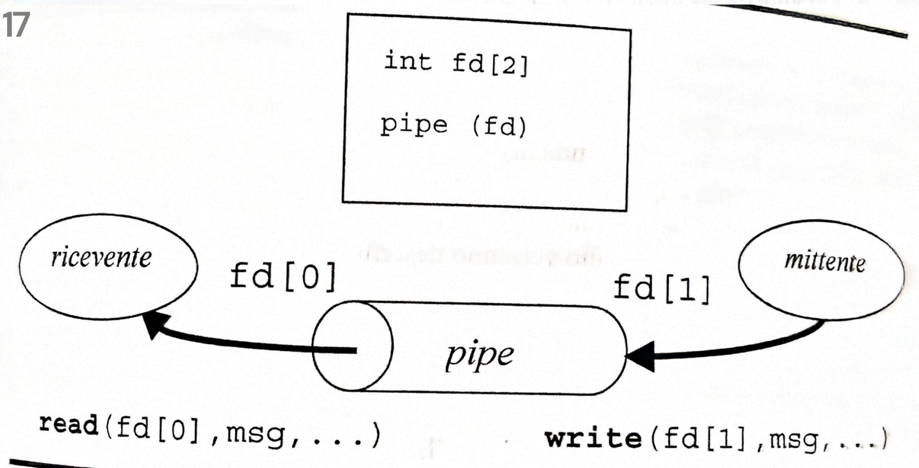


Figura 8.21 La pipe.